
Software Requirements Specification

for

SMART CLASSROOM MANAGEMENT SYSTEM

Version <X.X>

Prepared by

Group No: 27

Havishya.T	student 1	Se23uari098@mahindrauniversity.edu.in
Varshitha Reddy	student 2	Se23uari022@mahindrauniversity.edu.in
A.Chandrika	student 3	Se23uari002@mahindrauniversity.edu.in
K.V.Subba Rao	student 4	Se23uari068@mahindrauniversity.edu.in
Ch.Ramaraju	student 5	se23uari024@mahindrauniversity.edu.in
E.Hrishikeswar Reddy	student 6	se23uari036@mahindrauniversity.edu.in
Krushith Rao	student 7	se23ucse227@mahindrauniversity.edu.in
Vidhur Rao	student 8	se23ucse207@mahindrauniversity.edu.in

Instructor: Nartkannai K

Course: Software Engineering

Lab Section: *It block, 1st floor*

Date: 13-03-2026

Contents

CONTENTS	II
REVISIONS	II
1 INTRODUCTION.....	1

1.1	DOCUMENT PURPOSE.....	1
1.2	PRODUCT SCOPE	1
1.3	INTENDED AUDIENCE AND DOCUMENT OVERVIEW.....	1
1.4	DEFINITIONS, ACRONYMS AND ABBREVIATIONS	1
1.5	DOCUMENT CONVENTIONS	1
1.6	REFERENCES AND ACKNOWLEDGMENTS	2
2	OVERALL DESCRIPTION	2
2.1	PRODUCT OVERVIEW	2
2.2	PRODUCT FUNCTIONALITY	3
2.3	DESIGN AND IMPLEMENTATION CONSTRAINTS.....	3
2.4	ASSUMPTIONS AND DEPENDENCIES	3
3	SPECIFIC REQUIREMENTS	4
3.1	EXTERNAL INTERFACE REQUIREMENTS	4
3.2	FUNCTIONAL REQUIREMENTS.....	4
3.3	USE CASE MODEL	5
4	OTHER NON-FUNCTIONAL REQUIREMENTS.....	6
4.1	PERFORMANCE REQUIREMENTS	6
4.2	SAFETY AND SECURITY REQUIREMENTS.....	6
4.3	SOFTWARE QUALITY ATTRIBUTES	6
5	OTHER REQUIREMENTS	7
	APPENDIX A – DATA DICTIONARY	8
	APPENDIX B - GROUP LOG	9

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0 Draft	Smart Classroom Dev Team	Initial SRS draft – scope,functional requirements,use cases and non-functional requirements for Smart Classroom Management System.	13/03/2026

1 Introduction

This document is the Software Requirements Specification (SRS) for the Smart Classroom Management System (SCMS). The SCMS is a web-based platform designed to digitize and streamline the academic operations of a classroom — including activity and assignment management, digital attendance, grade tracking, timetabling, and AI-assisted learning.

1.1 Document Purpose

This SRS defines the complete software requirements for Version 1.0 of the Smart Classroom Management System. It establishes a formal agreement between the development team and stakeholders — including instructors, students, and academic administrators — regarding the features, behaviours, and constraints of the system. This document covers all functional subsystems of the SCMS: classroom activity management, assignment lifecycle, digital attendance, grade and performance tracking, timetable management, AI-powered suggestions, real-time collaboration, and a student self-service portal. It does not cover IoT hardware, physical sensor integration, or building automation.

1.2 Product Scope

The Smart Classroom Management System is a software-only platform that centralizes the academic workflow of instructors and students. It is designed to replace fragmented tools — separate grade books, paper attendance sheets, email-based assignment submission, and printed timetables — with a single integrated digital workspace.

The SCMS differs from existing ERP platforms such as Juno ERP by focusing exclusively on the classroom experience rather than institution-wide administrative operations. It introduces capabilities absent from Juno ERP: an AI-powered activity suggestion engine that recommends customized tasks based on student performance, real-time in-class collaboration tools, and a comprehensive student self-service portal where students independently manage their academic profile, submissions, and progress without instructor mediation.

Key benefits include reduced instructor administrative workload, increased student engagement through AI personalization and collaboration features, greater transparency via student self-service, and data-driven insights into classroom performance.

1.3 Intended Audience and Document Overview

- *This SRS is written for the following readers:*
- *Developers and Engineers — to design and implement the system based on specified requirements.*
- *Project Managers — to plan development milestones, sprints, and resource allocation.*
- *Quality Assurance Team — to derive test cases and define acceptance criteria.*
- *Instructors and Students (Client) — to validate that the system meets their academic needs.*
- *Course Instructor / Professor — to evaluate the completeness of this SRS document.*

The document is structured as follows: Section 1 introduces the project; Section 2 describes the overall system; Section 3 specifies all functional requirements and use cases; Section 4 covers non-functional requirements; Section 5 lists additional requirements; Appendices provide the data dictionary and group log.

1.4 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
AI	Artificial Intelligence
API	Application Programming Interface
COMET	Component-based Object and class Modeling with sequence diagrams and state machines — design methodology used in this project
ERP	Enterprise Resource Planning — e.g., Juno ERP (the comparable existing system)
GPA	Grade Point Average
LMS	Learning Management System
SCMS	Smart Classroom Management System — the product specified in this document
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language
SSP	Student Self-Service Portal — a key differentiator from Juno ERP
RBAC	Role-Based Access Control
WCAG	Web Content Accessibility Guidelines
GDPR	General Data Protection Regulation

1.5 Document Conventions

- This document follows IEEE 830-1998 formatting standards. The following conventions apply throughout:
- Font: Arial, 11–12pt for body text; larger sizes for section and sub-section headings as defined by the template.
- Line spacing: Single-spaced with 1-inch margins on all sides.
- Requirement IDs: Functional requirements are prefixed with F (e.g., F1, F2); performance with P; security with S; quality attributes with Q.
- Priority levels in requirement tables: High, Medium, or Low.
- Italicized text indicates instructional notes or template placeholders.
- Bold text denotes key terms, requirement IDs, or important identifiers.

1.6 References and Acknowledgments

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- Hassan Gomaa — Software Modeling and Design: UML, Use Cases, Patterns, and Software Architectures. Cambridge University Press, 2011. (COMET Method)

- *Juno ERP — Reference system studied for scope differentiation. (Internal reference)*
- *GDPR — General Data Protection Regulation, European Union, 2018.*
- *WCAG 2.1 — Web Content Accessibility Guidelines, W3C.*

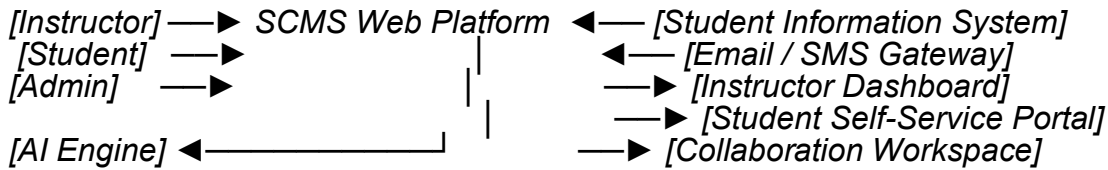
Overall Description

2.1 Product Overview

The Smart Classroom Management System is a new, self-contained web application that operates within a school or university's IT environment. It does not depend on any physical IoT sensors or hardware automation. All interactions take place through a browser-based interface accessible by instructors, students, and administrators on any device with an internet connection.

The SCMS sits between the institution's Student Information System (SIS) and the day-to-day classroom workflow. It pulls enrollment and timetable data from the SIS and pushes

attendance and grade summaries back, while providing a rich interactive workspace for both instructors and students.



2.2 Product Functionality

The following are the major functions of the Smart Classroom Management System:

- **Assignment and Activity Management** — Instructors create, distribute, and manage class assignments, quizzes, and activities. Students submit work digitally and receive feedback through the platform.
- **Digital Attendance Tracking** — Instructors mark attendance manually through the dashboard; students can check their own attendance record through the self-service portal.
- **Grades and Performance Tracking** — Instructors enter and manage grades; the system calculates scores, GPA, and progress metrics. Students view their own performance analytics in real time.
- **Timetable and Scheduling** — Instructors and administrators manage class timetables, room bookings, and academic calendars within the system.
- **AI-Powered Activity Suggestions** — The system analyzes student performance data and recommends personalized activities and remedial exercises to instructors, differentiating it from Juno ERP.
- **Real-Time Collaboration Tools** — Students and instructors collaborate on shared documents, discussion boards, and live Q&A sessions within the platform, a feature absent from Juno ERP.
- **Student Self-Service Portal** — Students independently access and manage their assignments, attendance history, grades, timetable, and submission deadlines without requiring instructor intervention, going beyond Juno ERP's student-facing capabilities.
- **Notification and Alerts** — Automated notifications are sent to students and instructors for assignment deadlines, grade updates, attendance warnings, and AI-generated recommendations.

2.3 Design and Implementation Constraints

- The system shall be designed using the COMET method. All design artifacts must use UML notation.
- The backend must expose a RESTful API. The frontend must be a responsive web application compatible with Chrome, Firefox, and Edge (latest two major versions).
- The system is purely software-based and must not require any physical IoT devices, sensors, or classroom hardware to function.
- A relational database (e.g., PostgreSQL) shall be used for all structured data including users, grades, attendance, and assignments.
- The AI activity suggestion module must use student performance data already stored in the system and must not require external paid AI API calls for core functionality (open-source ML libraries preferred).
- All student academic data, grades, and performance records must be stored encrypted and accessed only by authorized users per RBAC rules.

- The web interface must conform to WCAG 2.1 Level AA accessibility guidelines.

2.4. Assumptions and Dependencies

- It is assumed that the institution's Student Information System (SIS) provides a documented API or data export for student enrollment and timetable data.
- Instructors and students have valid institutional email accounts used for login and notifications.
- All users access the system through a standard web browser on a device with a reliable internet connection.
- The institution has an email/SMS gateway available for notification delivery.
- The AI suggestion engine operates on historical grade and activity completion data; meaningful recommendations require at least one prior graded activity per student.
- A mobile companion app is out of scope for Version 1.0 but the API architecture must support it in a future release.

Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- **Instructor Dashboard:** A browser-based interface allowing instructors to create and manage assignments and activities, take attendance, enter and view grades, manage the class timetable, view AI activity suggestions, monitor collaboration sessions, and generate class reports. Interaction is via mouse and keyboard on desktop and touch on tablet.
- **Student Self-Service Portal:** A student-facing interface where students independently view and submit assignments, check attendance records, view grades and progress analytics, access the class timetable, and participate in collaboration tools — without needing to contact the instructor for routine information.
- **Administrator Portal:** A configuration and oversight interface for managing user accounts, class enrollments, institutional timetables, system-wide reports, and notification settings.
- **Notifications Panel:** An in-app notification feed visible to all users displaying deadline reminders, grade updates, attendance alerts, and AI recommendations.

3.1.2 Hardware Interfaces

The SCMS is a purely software-based system and does not interface with any physical hardware, IoT sensors, or classroom devices. The only hardware assumption is a standard internet-connected computing device (desktop, laptop, or tablet) used to access the web application.

3.1.3 Software Interfaces

- **Student Information System (SIS):** Read-only REST API integration to import student enrollment lists, class rosters, and base timetable data.
- **Email Gateway (SMTP):** Integration with the institution's SMTP server for sending automated email notifications for deadlines, grade releases, and attendance alerts.
- **SMS Gateway (Optional):** Integration with an SMS API (e.g., Twilio) for critical alerts such as attendance warnings.
- **AI Recommendation Engine:** An internal service deployed as a microservice that reads from the SCMS database and returns personalized activity suggestions; exposed as an internal REST API.
- **Student Information System (SIS):** Read-only REST API integration to import student enrollment lists, class rosters, and base timetable data.
- **Email Gateway (SMTP):** Integration with the institution's SMTP server for sending automated email notifications for deadlines, grade releases, and attendance alerts.
- **SMS Gateway (Optional):** Integration with an SMS API (e.g., Twilio) for critical alerts such as attendance warnings.
- **AI Recommendation Engine:** An internal service deployed as a microservice that reads from the SCMS database and returns personalized activity suggestions; exposed as an internal REST API.

3.2 Functional Requirements

The table below details the functional requirements of SCMS. Requirements are grouped by feature area and prioritized.

Req.ID	Description	Priority
Assignment & Activity Management :		
F1	The system shall allow instructors to create assignments with a title, description, due date, maximum marks, and optional file attachment.	High
F2	The system shall allow instructors to create in-class activities (e.g., quizzes, group tasks, discussions) and assign them to one or more classes.	High
F3	The system shall allow students to submit assignments digitally by uploading files or entering text through the student portal before the deadline.	High
F4	The system shall prevent submission after the due date unless the instructor has granted an extension for the specific student.	Medium

F5	The system shall allow instructors to review student submissions, annotate them, and enter a grade and feedback comment directly on the platform.	High
F6	The system shall notify students via email and in-app notification 24 hours before an assignment deadline.	Medium
F7	The system shall display a submission status (Not Submitted, Submitted, Graded) for each assignment visible to both the student and instructor.	High
Attendance Tracking:		
F8	The system shall allow instructors to mark each enrolled student as Present, Absent, or Late for any class session through the dashboard.	High
F9	The system shall allow instructors to edit a previously submitted attendance record within 48 hours of the session.	Medium
F10	The system shall automatically calculate and display each student's attendance percentage for each subject.	High
F11	The system shall send an automated attendance warning notification to a student and their assigned guardian email when attendance drops below a configurable threshold (default: 75%).	High
F12	Students shall be able to view their own attendance record — per subject and per date — through the student self-service portal.	High
Grades & Performance Tracking:		
F13	The system shall allow instructors to enter and	High

	update marks for each assignment and activity.	
F14	The system shall automatically compute the cumulative grade and GPA for each student based on all graded items, using a configurable grading scale.	High
F15	The system shall provide instructors with a class-level performance summary showing average scores, grade distribution, and students below a threshold.	Medium
F16	Students shall be able to view their grades, marks per assignment, cumulative score, and a personal performance trend graph through the self-service portal.	High
F17	The system shall generate a grade report per class exportable as PDF and CSV.	Medium
<i>Timetable & Scheduling:</i>		
F18	The system shall allow administrators to create and manage class timetables, specifying subject, instructor, room, day, and time for each session.	High
F19	The system shall display the current day's schedule to instructors and students on their respective dashboards upon login.	High
F20	The system shall detect and flag scheduling conflicts (same instructor, same room, or same student group assigned at the same time) when a timetable is being created or modified.	High
F21	Students shall be able to view their full weekly timetable through the self-service portal.	Medium
<i>AI-Powered Activity Suggestions — Key</i>		

<i>Differentiator from Juno ERP:</i>		
<i>F22</i>	<i>The system shall analyze each student's historical grade and activity completion data and generate a ranked list of recommended remedial or enrichment activities for the instructor's review.</i>	<i>High</i>
<i>F23</i>	<i>The AI suggestion engine shall categorize each recommendation as Remedial (student performing below class average), On-Track, or Enrichment (student performing above class average).</i>	<i>High</i>
<i>F24</i>	<i>Instructors shall be able to accept, modify, or dismiss AI activity suggestions directly from the dashboard.</i>	<i>Medium</i>
<i>F25</i>	<i>The system shall display a plain-language explanation for each AI suggestion (e.g., "Student scored below 50% in last 3 quizzes on Topic X") so the instructor can make an informed decision.</i>	<i>Medium</i>
<i>Real-Time Collaboration Tools — Key Differentiator from Juno ERP:</i>		
<i>F26</i>	<i>The system shall provide a per-class discussion board where instructors and students can post, reply to, and like messages.</i>	<i>High</i>
<i>F27</i>	<i>The system shall allow instructors to create a live Q&A session linked to a class, where students submit questions in real time and the instructor responds.</i>	<i>High</i>
<i>F28</i>	<i>The system shall support collaborative document editing for group assignments, allowing multiple students to edit a</i>	<i>Medium</i>

	<i>shared document simultaneously.</i>	
<i>F29</i>	<i>All collaboration activity (posts, replies, Q&A questions) shall be timestamped and stored for later review.</i>	<i>Low</i>
<i>Student Self-Service Portal — Key Differentiator from Juno ERP:</i>		
<i>F30</i>	<i>Students shall be able to log in and independently access their timetable, assignments, grades, attendance records, and notifications without instructor assistance.</i>	<i>High</i>
<i>F31</i>	<i>Students shall be able to submit re-grade requests through the portal, providing a written justification; instructors will be notified of the request.</i>	<i>Medium</i>
<i>F32</i>	<i>Students shall be able to update their contact information and notification preferences through the portal.</i>	<i>Low</i>
<i>Access Control & Administration</i>		
<i>F33</i>	<i>The system shall implement role-based access control with four roles: Administrator, Instructor, Student, and Guardian.</i>	<i>High</i>
<i>F34</i>	<i>The system shall allow administrators to create user accounts, assign roles, enroll students in classes, and deactivate accounts.</i>	<i>High</i>
<i>F35</i>	<i>The system shall maintain a full audit log of all grade modifications, attendance edits, and administrative actions with timestamps and the user who performed each action.</i>	<i>Medium</i>

3.3 Use Case Model

3.3.1 Use Case U1 — Manage and Submit Assignment

<i>Author</i>	<i>Smart Classroom Dev Team</i>
<i>Purpose</i>	<i>To allow instructors to create and distribute assignments to students, and allow students to submit their work digitally through the student self-service portal.</i>
<i>Requirements Traceability</i>	<i>F1, F2, F3, F4, F5, F6, F7</i>
<i>Priority</i>	<i>High</i>
<i>Preconditions</i>	<i>Instructor is logged in and has at least one active class. Student is enrolled in the class and logged into the student portal.</i>
<i>Post Conditions</i>	<i>Assignment is published and visible to all enrolled students. Upon submission, status is updated to "Submitted". Upon grading, status changes to "Graded" and student is notified.</i>
<i>Actors</i>	<i>Instructor (creates and grades), Student (submits), System (sends notifications)</i>
<i>Extends</i>	<i>None</i>
<i>Flow of Events</i>	<i>Basic Flow — Instructor creates assignment: 1. Instructor navigates to Assignments and clicks Create New Assignment. 2. Instructor fills in title, description, due date, and maximum marks; optionally attaches a resource file. 3. Instructor selects the target class(es) and clicks Publish. 4. System saves the assignment, updates student dashboards, and sends a deadline notification to all enrolled students. Basic Flow — Student submits assignment: 1. Student logs into the self-service portal and opens the assignment. 2. Student uploads a file or enters a text response and clicks Submit. 3. System records the submission with a timestamp and updates the status to "Submitted". 4. Instructor is notified of the new submission and reviews it on the dashboard. 5. Instructor enters marks and feedback; system notifies student that grading is complete. Alternative Flow — Late submission with extension: 1. Instructor grants a deadline extension for a specific student from the assignment settings. 2. Student may submit after original due date; submission is accepted</i>

	and marked "Extended Submission". Exceptions: Student attempts to submit after deadline without extension — system rejects submission and displays a message directing the student to request an extension. File upload fails — system displays error and prompts student to retry or use text submission.
Includes	U4 (Send Notification)
Notes/Issues	File size limit for submissions must be configurable by the administrator. Supported file types should include PDF, DOCX, and images.

3.3.2 Use Case U2 — Mark and Monitor Attendance

Author	Smart Classroom Dev Team
Purpose	To allow instructors to create and distribute assignments to students, and allow students to submit their work digitally through the student self-service portal.
Requirements Traceability	F8, F9, F10, F11, F12
Priority	High
Preconditions	A class session is scheduled in the timetable. Students are enrolled in the class. Instructor is logged in.
Post Conditions	Attendance is recorded for all students. Attendance percentages are updated. Warning notifications are sent if thresholds are breached.
Actors	Instructor (marks attendance), Student (views record), System (calculates percentage, sends warnings)
Extends	None
Flow of Events	Basic Flow: 1. Instructor opens the Attendance section and selects the current class session. 2. System displays the full enrolled student list with options: Present, Absent, Late. 3. Instructor marks each student and clicks Save. 4. System stores the record, recalculates each student's attendance percentage, and checks against threshold. 5. For students below the threshold, system sends warning notifications to the student and guardian email. Alternative Flow — Edit attendance: 1. Instructor revisits a past session within 48 hours, modifies a student's status, and saves. 2. System logs the change in the audit trail with the editor's name and

	<i>timestamp. Alternative Flow — Student views attendance: 1. Student logs into the self-service portal and navigates to My Attendance. 2. System displays a table of dates and statuses per subject along with a cumulative attendance percentage. Exceptions: Instructor attempts to edit attendance after 48 hours — system blocks the edit and prompts the instructor to contact an administrator.</i>
<i>Includes</i>	<i>U4 (Send Notification)</i>
<i>Notes/Issues</i>	<i>The attendance threshold percentage must be configurable by the administrator per institution policy. Default is 75%.</i>

3.3.3 Use Case U3 — AI Activity Suggestion

<i>Author</i>	<i>Smart Classroom Dev Team</i>
<i>Purpose</i>	<i>To provide instructors with AI-generated personalized activity recommendations for individual students based on their academic performance, differentiating the SCMS from Juno ERP.</i>
<i>Requirements Traceability</i>	<i>F22, F23, F24, F25</i>
<i>Priority</i>	<i>High</i>
<i>Preconditions</i>	<i>At least one graded assignment or activity exists for students in the class. AI Engine has processed available performance data</i>
<i>Post Conditions</i>	<i>Instructor reviews and acts on AI suggestions. Accepted suggestions are assigned to students as activities.</i>
<i>Actors</i>	<i>AI Engine (generates recommendations), Instructor (reviews and acts)</i>
<i>Extends</i>	<i>U1 (Manage and Submit Assignment) — when instructor accepts a suggestion, it becomes a new activity.</i>
<i>Flow of Events</i>	<i>Basic Flow: 1. AI Engine periodically (daily or on-demand) analyzes grade and activity completion data for each student in a class. 2. Engine categorizes each student as Remedial, On-Track, or Enrichment and generates recommended activities. 3. Instructor receives a notification: "AI suggestions available for your class." 4. Instructor opens the AI Suggestions panel; each suggestion shows the student's name, category, recommended activity, and a plain-language explanation. 5. Instructor accepts a suggestion — system creates a new assignment pre-filled with the suggested activity details. 6. Instructor</i>

	<i>dismisses a suggestion — system logs the dismissal and does not re-suggest the same activity within 7 days. Alternative Flow — Instructor modifies suggestion: 1. Instructor clicks Edit on a suggestion, modifies the activity title or description, then accepts. Exceptions: Insufficient data — AI engine shows "Not enough data to generate suggestions. At least one graded item is required."</i>
<i>Includes</i>	<i>U4 (Send Notification)</i>
<i>Notes/Issues</i>	<i>AI suggestions are advisory only; instructors have full authority to override or dismiss them. The AI engine must be explainable — no black-box recommendations without reasons.</i>

3.3.4 Use Case U4 — Real-Time Collaboration (Discussion Board and Live Q&A)

<i>Author</i>	<i>Smart Classroom Dev Team</i>
<i>Purpose</i>	<i>To facilitate real-time academic interaction between instructors and students through a class discussion board and live Q&A sessions, a feature not available in Juno ERP.</i>
<i>Requirements Traceability</i>	<i>F26, F27, F28, F29</i>
<i>Priority</i>	<i>High</i>
<i>Preconditions</i>	<i>User is logged in and a class is active. For Live Q&A, the instructor must initiate the session.</i>
<i>Post Conditions</i>	<i>Discussion posts and Q&A exchanges are stored and accessible for future reference.</i>
<i>Actors</i>	<i>Instructor (creates sessions, responds), Student (posts, asks questions)</i>
<i>Extends</i>	<i>None</i>
<i>Flow of Events</i>	<i>Basic Flow — Discussion Board: 1. Student or instructor navigates to the Discussion Board for a class. 2. User creates a new post with a title and message body. 3. Other users in the class can reply or like the post; all enrolled members receive an in-app notification. Basic Flow — Live Q&A: 1. Instructor starts a Live Q&A session from the class dashboard. 2. Students receive a notification and join the session. 3. Students submit questions in real time; questions appear in a live feed visible to the instructor. 4. Instructor responds to each question; response is broadcast to all</i>

	session participants. 5. Instructor ends the session; a full transcript is saved and accessible from the class page. <i>Exceptions: Network interruption during Live Q&A — system buffers messages locally and syncs when connection resumes.</i>
<i>Includes</i>	<i>None</i>
<i>Notes/Issues</i>	<i>All content posted in collaboration tools must be subject to a moderation flag — instructors can hide or delete inappropriate posts.</i>

Other Non-functional Requirements

4.1 Performance Requirements

<i>ID</i>	<i>Requirement</i>
<i>P1</i>	<i>All standard page loads (dashboard, assignment list, grade view) shall complete within 3 seconds on a standard broadband connection.</i>
<i>P2</i>	<i>Assignment submission uploads of up to 25 MB shall complete within 10 seconds.</i>
<i>P3</i>	<i>Attendance records shall be saved and reflected on the instructor's dashboard within 2 seconds of clicking Save.</i>
<i>P4</i>	<i>The AI Engine shall generate and return activity suggestions within 15 seconds of a manual refresh request by the instructor.</i>
<i>P5</i>	<i>Live Q&A messages shall be delivered to all participants within 1 second of submission under normal network conditions.</i>
<i>P6</i>	<i>The system shall support up to 500 concurrent users (across all roles) without performance degradation.</i>
<i>P7</i>	<i>PDF and CSV grade reports for a single class semester shall generate within 15 seconds</i>

4.2 Safety and Security Requirements

<i>ID</i>	<i>Requirement</i>
<i>S1</i>	<i>All data transmitted between clients and the server shall be encrypted using HTTPS (TLS 1.2 or higher).</i>
<i>S2</i>	<i>Passwords shall be stored using a strong one-way hashing algorithm (e.g., bcrypt with a minimum cost factor of 12).</i>
<i>S3</i>	<i>The system shall enforce RBAC — students may not access other students'</i>

	<i>grades or submissions; instructors may not access classes they are not assigned to.</i>
S4	<i>User sessions shall automatically expire after 30 minutes of inactivity.</i>
S5	<i>After 5 consecutive failed login attempts, the account shall be temporarily locked for 15 minutes and the registered email notified.</i>
S6	<i>Grade modifications shall be recorded in a tamper-evident audit log capturing the original value, new value, timestamp, and the user who made the change.</i>
S7	<i>Student academic data shall not be shared with or accessible by any third-party service without explicit administrator authorization.</i>
S8	<i>The system shall comply with applicable data protection regulations (e.g., GDPR) regarding student data storage, access, and the right to erasure.</i>

4.3 Software Quality Attributes

4.3.1 Reliability

Q1. The SCMS shall maintain a minimum uptime of 99.5% during active academic hours (7:00 AM – 10:00 PM on weekdays). To achieve this, the system shall be hosted on a cloud platform with auto-scaling and shall implement automated health checks with restart capabilities. In the event of a database failure, the system shall switch to a read-only mode displaying last-known data rather than going completely offline.

4.3.2 Maintainability

Q2. The system shall follow a modular, microservices-based architecture where each subsystem (Assignments, Attendance, Grades, Timetable, AI Engine, Collaboration) is a separately deployable service. This allows individual components to be updated, patched, or replaced without taking the entire platform offline. All services shall have corresponding unit and integration tests with a minimum code coverage of 80%.

4.3.3 Scalability and Design for Change

Q3. The system shall be designed to accommodate future additions without restructuring the core platform. Specifically: (a) the AI suggestion engine can be swapped for a more advanced model by updating the internal AI API contract; (b) new activity types can be added by extending the Activity schema without modifying existing activity logic; (c)

additional notification channels (e.g., WhatsApp, push notifications) can be integrated through a pluggable notification adapter pattern.

4.3.4 Usability

Q4. The system shall be designed so that a first-time user with no prior training can complete core tasks — viewing today's timetable, marking attendance, and creating an assignment — within 10 minutes of first login. The interface shall follow Nielsen's 10 usability heuristics. User acceptance testing shall be conducted with at least 5 representative instructors and 5 students, targeting a System Usability Scale (SUS) score of 75 or above before release.

4.3.5 Portability

Q5. The SCMS backend shall be containerized using Docker and deployable on any major cloud platform (AWS, Azure, GCP) or an on-premise Linux server without modification to the application code. The frontend shall function correctly on Chrome, Firefox, and Edge (latest two major versions) on both desktop and tablet screen sizes.

Other Requirements

- **Database:** All structured data (users, grades, attendance, assignments, timetables) shall be stored in PostgreSQL 15 or higher. Full database backups shall run daily; incremental backups every 4 hours. Recovery Point Objective (RPO): 4 hours. Recovery Time Objective (RTO): 2 hours.
- **Internationalization:** Version 1.0 shall support English only. The codebase shall use an i18n resource file architecture to enable translation into other languages in future versions without code changes.
- **Legal and Compliance:** The system shall include a consent acknowledgment screen for new users outlining how their academic data is stored and used. Students shall be able to request data export or deletion through the self-service portal in accordance with applicable data protection laws.
- **Audit and Logging:** All user actions — login events, grade entries, attendance edits, and administrative changes — shall be written to a centralized audit log accessible only to Administrators.
- **Browser Notifications:** In addition to email, the system shall support web push notifications for deadline reminders and live Q&A session start alerts, with users able to opt in or out through their notification preferences.

Appendix A – Data Dictionary

<i>Name</i>	<i>Type</i>	<i>Description</i>	<i>Related Operations</i>	<i>Req. Ref.</i>
<i>StudentID</i>	<i>String (Input)</i>	<i>Unique student identifier imported from the SIS</i>	<i>Login, Attendance, Grades</i>	<i>F8, F13, F30</i>
<i>AssignmentStatus</i>	<i>Enum {Not Submitted, Submitted, Graded, Late}</i>	<i>Current submission state of a student's assignment</i>	<i>Submit Assignment, Grade Assignment</i>	<i>F3, F4, F5, F7</i>
<i>AttendanceStatus</i>	<i>Enum {Present, Absent, Late}</i>	<i>Attendance state per student per session</i>	<i>Mark Attendance, View Record</i>	<i>F8, F9, F12</i>
<i>AttendancePct</i>	<i>Float (%) — Computed</i>	<i>Percentage of sessions attended out of total held, per student per subject</i>	<i>Attendance Warning</i>	<i>F10, F11</i>
<i>ATTEND_THRESHOLD</i>	<i>Constant: 75% (configurable)</i>	<i>Minimum attendance percentage; breach triggers warning notification</i>	<i>Attendance Warning</i>	<i>F11</i>
<i>Grade</i>	<i>Float — Computed</i>	<i>Cumulative grade point average calculated from all graded items using the configured grading scale</i>	<i>Enter Grade, View Grade, GPA Calc</i>	<i>F13, F14, F16</i>
<i>GPA</i>	<i>Float — Computed</i>	<i>Cumulative grade point average calculated from all graded items using the configured grading scale</i>	<i>View Performance</i>	<i>F14, F16</i>
<i>AI Suggestion</i>	<i>Object (Output)</i>	<i>AI recommendation: studentID, category (Remedial / On-Track / Enrichment), suggested</i>	<i>AI Engine, Instructor Review</i>	<i>F22, F23, F24, F25</i>

		<i>activity, explanation text</i>		
<i>ClassSession</i>	<i>Object (State)</i>	<i>Scheduled class record: subject, instructor, room, day, time, enrolled students</i>	<i>Timetable, Attendance, Collaboration</i>	<i>F18, F19, F20</i>
<i>DiscussionPost</i>	<i>Object (State)</i>	<i>Collaboration post: authorID, classID, body, timestamp, replies list</i>	<i>Discussion Board</i>	<i>F26, F29</i>
<i>QASession</i>	<i>Object (State)</i>	<i>Live Q&A session: classID, instructorID, start time, end time, questions list</i>	<i>Live Q&A</i>	<i>F27, F29</i>
<i>UserRole</i>	<i>Enum {Administrator, Instructor, Student, Guardian}</i>	<i>Role assigned to user; governs RBAC access permissions</i>	<i>Login, Access Control</i>	<i>F33, F34</i>
<i>AuditLogEntry</i>	<i>Object (State)</i>	<i>Tamper-evident record: action type, actor userID, timestamp, before value, after value</i>	<i>Audit Trail</i>	<i>F35, S6</i>
<i>Notification</i>	<i>Object (Output)</i>	<i>Message sent to user: type, recipient, channel (email / in-app / SMS), message body, timestamp</i>	<i>Deadline Alert, Attendance Warning, AI Alert</i>	<i>F6, F11, F23</i>

Appendix B - Group Log

<i>Date</i>	<i>Attendees</i>	<i>Activities / Decision</i>	<i>Next Steps</i>
<i>DD/MM/YYYY</i>	<i>All members</i>	<i>Project kickoff. Reviewed project scope. Confirmed system is software-only with no IoT. Agreed on key</i>	<i>Research LMS APIs and AI recommendation approaches.</i>

		<i>differentiators from Juno ERP: AI suggestions, collaboration tools, student self-service portal. Assigned research tasks.</i>	
<i>DD/MM/YYYY</i>	<i>All members</i>	<i>Completed Section 1 and Section 2. Drafted product overview and functionality list. Confirmed feature set with instructor.</i>	<i>Draft all functional requirements.</i>
<i>DD/MM/YYYY</i>	<i>All members</i>	<i>Completed non-functional requirements F1–F35. Drafted use cases U1–U4. Reviewed use case flows for completeness.</i>	<i>Complete non-functional requirements and appendices.</i>
<i>DD/MM/YYYY</i>	<i>All members</i>	<i>Completed Sections 4 and 5 and both appendices. Full document review and proofreading. Formatted per template. Ready for submission.</i>	<i>Submit SRS document.</i>